```
    const newTickets = Array.from(
      Array(state.ticketsToBuyCount).keys()
    ).map(
»     (index): TicketData => {
        return {
          id: 0,
          row: action.rowNumber,
          number: action.seatNumber + index,
          status: "held",
        }
      }
    )
    return {
      ...state,
      myTickets: [...state.myTickets, ...newTickets],
    }
  }
```

As a reminder, what the code here is doing is that when a holdTicket action is called, it's taking the row/seat combination that triggered the call and creating a new TicketData object for that seat and however many seats to the right are needed based on the "How Many Tickets" pull-down menu.

The catch in the new code comes from hard-coding the status as "held" without further specifying the type. TypeScript will infer that "held" is a string but won't infer by itself that the expected type is TypeStatus, even if "held" is a valid value for TypeStatus. The solution is to explicitly specify that the return value is a TicketData object, which also specifies that the status value is a TypeStatus, which causes TypeScript to validate that assignment correctly. You can confirm that the validation is happening by changing that "held" value to something that isn't one of the five literal values and see that TypeScript throws a type error.

Similarly, we can specify a few cases in the Seat component file to take our new TicketStatus type rather than a string. After importing the TicketStatus type, change the type of the status argument in the stateColor function, and also in the SquareProps interface declaration and the return value of the currentStatus function.

In addition to string literal types, TypeScript also allows you to have numeric literal types: